

Assignment 2: Super Tic-Tac-Toe

Reinforcement Learning

MAFS5370 Reinforcement Learning with Applications to Finance

RONG Zhixia Student ID: 21269362

Executive summary. This project implements and evaluates a reinforcement learning agent for the stochastic Super Tic-Tac-Toe game. The solution first validates the simulator on small 4 x 4 boards and then trains a full-board linear Q-learning agent on the 96-cell triangular board. The final version uses mixed-opponent training, tactical safeguards for immediate wins and blocks, and explicit tests against both random and heuristic opponents.

Full game board

96 cells

six 4 x 4 squares in three levels

vs random opponent

100%

win rate as both first and second player

vs heuristic opponent

45.8%

win rate as first player after mixed training

Report component	What is demonstrated
Game environment	A coordinate-based 96-cell board, stochastic placement, and row / column / diagonal win checking.
RL formulation	State, action, transition, reward and episodic objective for a two-player stochastic game.
Learning methods	Small-board tabular Q-learning and scalable full-board linear Q-learning.
Evaluation	Training curves, final win/draw/loss rates, first / second player tests, and feature interpretation.

Key message

The trained agent is a strong and interpretable RL baseline. It reliably exploits random opponents, improves against heuristic opponents after mixed training, and still reveals a clear limitation: defensive play remains weaker than offensive line-building.

Design choice	Reason
Triangular board embedded in 2D	Allows systematic generation of rows, columns and diagonal windows.
Linear Q on full board	Avoids an infeasible tabular state space on the 96-cell board.
Random and heuristic evaluation	Distinguishes easy random-play performance from stronger tactical robustness.

Submission-ready outputs

The report is designed to accompany the executable Jupyter notebook. The notebook produces the convergence curve, final evaluation table, feature-weight diagnostics and board-layout figure used below. This creates a reproducible link between modelling, code and reported results.

1. Game Environment and Rule Implementation

The Super Tic-Tac-Toe board contains six 4 x 4 squares arranged as a triangle. To make rule checking systematic, the six squares are embedded into a two-dimensional coordinate system. Each playable cell stores its id, coordinate, level and block. This representation allows row, column and diagonal windows to be generated from coordinates while retaining level information for the special column rule.

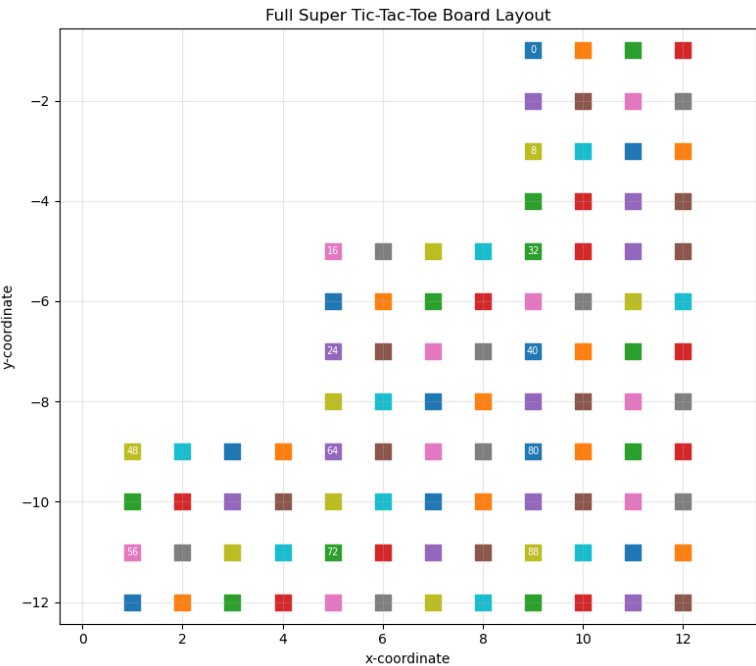


Figure 1. Embedded full Super Tic-Tac-Toe board layout. The figure shows the triangular 96-cell board used by the simulator.

Component	Implementation
Playable cells	96 cells: 16 on level 1, 32 on level 2, and 48 on level 3.
Action	The active player selects an empty cell. Board values are 0 for empty, 1 for learner and -1 for opponent.
Noisy placement	Probability 1/2 on the chosen cell; probability 1/16 on each of the 8 neighbouring cells. Invalid or occupied targets forfeit the move.
Winning condition	4 in a row, 4 in a column, or 5 across a diagonal. A column win must cross at least two levels.

Environment validation

The implementation validates board construction, noisy placement, row / column / diagonal wins, and the column-cross-level rule through unit-test style checks. This reduces the risk that the learning results come from a mis-specified game environment.

2. Reinforcement Learning Formulation

The task is formulated as an episodic two-player stochastic game. The learner is trained against controlled opponent policies and evaluated as both the first and second player. Stochasticity comes from the noisy placement rule, so the same selected action may lead to different board states.

Element	Definition
State	$s_t = (B_t, p_t)$, where B_t is the current board and p_t is the current player.
Action	a_t is an empty playable cell selected by the active player.
Transition	Noisy placement, legality checking, win checking and player switching.
Reward	+1 for learner win, -1 for learner loss, 0 for draw or non-terminal transition; shaping is used in full-board training.
Objective	Maximise expected discounted return over an episode.

3. Learning Algorithms

Tabular Q-learning baseline

For small boards, Q-values are stored explicitly. The update is $Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$. These baselines validate the training loop but do not scale to the 96-cell game.

Full-board linear Q-learning

For the full triangular board, $Q_{\theta}(s,a) = \theta^T \phi(s,a)$. Features include centrality, level indicators, open-line strength, immediate-win signals, blocking signals and board occupancy.

Experiment	Board	Method	Purpose
base_deterministic_tabular	4 x 4, no noise	Tabular Q-learning	Simple baseline and sanity check.
base_noisy_tabular	4 x 4, noisy placement	Tabular Q-learning	Robustness under stochastic placement.

Experiment	Board	Method	Purpose
full_super_linear_mixed	96-cell triangular board	Linear Q-learning + mixed opponent	Main full-board agent.

The final full-board policy uses mixed-opponent training and tactical safeguards: the agent first takes an immediate winning move if available, then blocks an opponent immediate win if necessary, and otherwise follows the learned linear Q policy.

4. Training Behaviour

The convergence plot records evaluation win rate during training. The small-board tabular baselines remain unstable and do not reach a strong threshold. In contrast, the full-board linear mixed agent rapidly reaches near-perfect performance against a random opponent, showing the benefit of feature-based value approximation in the large board.

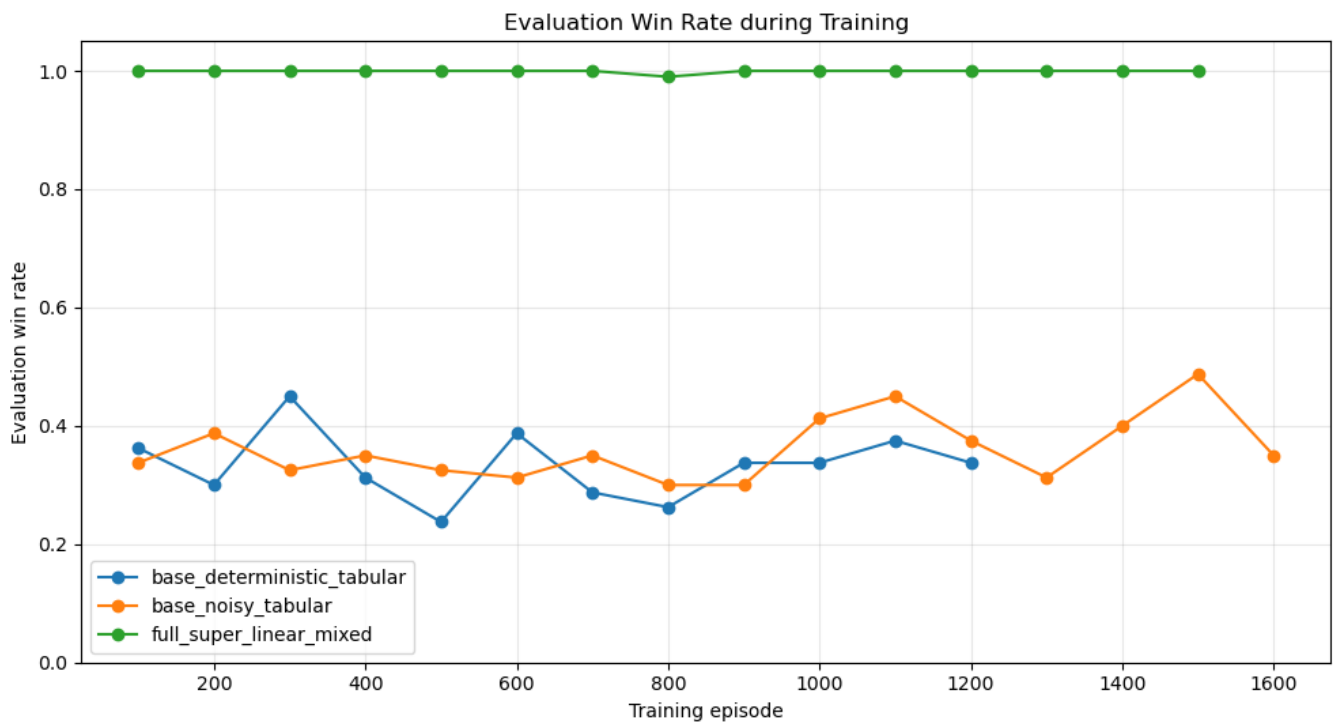


Figure 2. Evaluation win rate during training. The full-board agent is evaluated against a random opponent.

Experiment	Final / best win rate	Interpretation
base deterministic tabular	0.3375 / 0.4625	Naive tabular learning gives only moderate and unstable performance.
base noisy tabular	0.3375 / 0.4375	Noisy placement makes the baseline less reliable.
full super linear mixed	1.0000 / 1.0000	The feature-based full-board agent learns a strong policy against random play.

5. Final Evaluation and Policy Interpretation

The final evaluation compares the trained full-board agent with random and heuristic opponents. The random-vs-random baseline is included to separate learned performance from move-order advantage.

Agent / baseline	Opponent	Role	Win	Draw	Loss
full_super_linear_mixed	random	first	1.000	0.000	0.000
full_super_linear_mixed	random	second	1.000	0.000	0.000
full_super_linear_mixed	heuristic	first	0.458	0.000	0.542
full_super_linear_mixed	heuristic	second	0.366	0.000	0.634
random_vs_random baseline	random	first	0.490	0.000	0.510

Against random play

The full-board agent wins 100% as both first and second player. Compared with the random-vs-random baseline of 49.0%, this indicates clear learning rather than first-move advantage.

Against heuristic play

The win rate falls to 45.8% as first player and 36.6% as second player. The policy is strong against weak opponents but still improvable against tactical play.

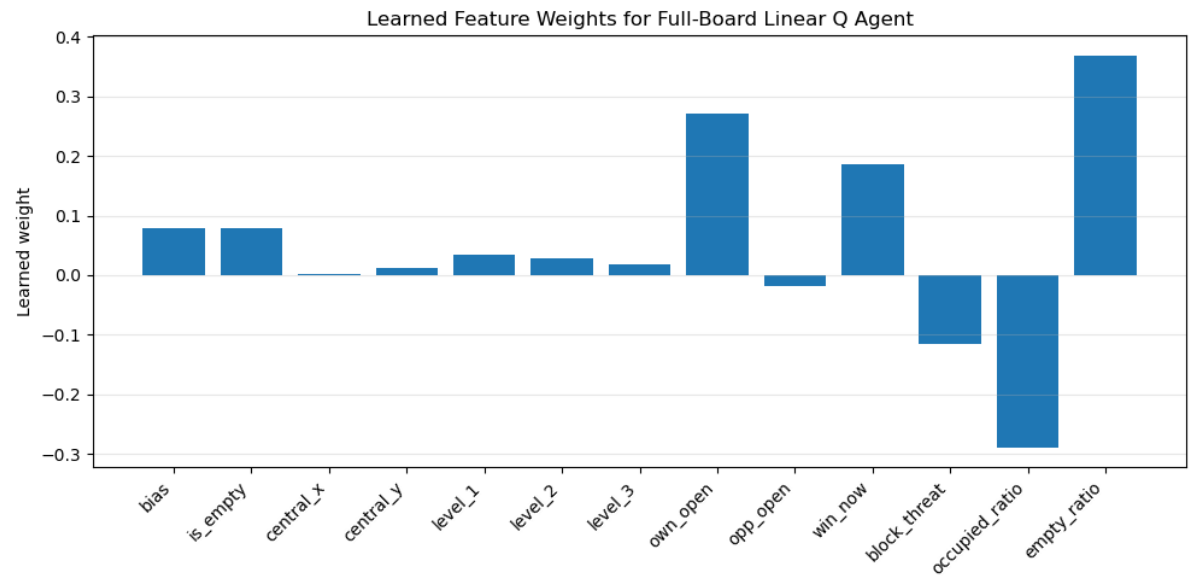


Figure 3. Learned feature weights for the full-board linear Q agent.

6. Discussion, Limitations and Conclusion

The learned weights show that the agent is mainly offensive. Positive weights on `own_open` and `win_now` mean that it builds and completes its own winning lines. The negative `block_threat` coefficient indicates that defensive learning is weaker, although tactical safeguards partly compensate. This explains the strong random-opponent results and the weaker performance against the heuristic opponent.

Feature group	Observed result	Interpretation
own_open and win_now	Positive and relatively large	The agent actively builds and completes its own winning lines.
block_threat	Negative in the linear weights	The linear component is not strongly defensive; this limits heuristic-opponent performance.
empty / occupied ratios	empty positive, occupied negative	The policy prefers earlier board states with more available action space.

Strengths

Strength	Explanation
Validated environment	The board construction, stochastic placement and win checking are explicit and testable.
Scalable full-board method	Linear Q-learning avoids the infeasible tabular state space on the 96-cell board.
Robust evaluation design	The final evaluation uses random and heuristic opponents and tests both player roles.
Interpretable output	Feature weights explain why the agent is strong against random play but weaker defensively.

Limitations and future work

Limitation	Implication
Random opponent is weak	Perfect random-opponent results should not be interpreted as solving the game.
Heuristic opponent remains challenging	The agent still loses more than half of games against heuristic play, especially as second player.
Future extensions	Increase heuristic-opponent training ratio, tune blocking rewards, use self-play, or implement DQN / policy-gradient methods for the bonus track.

Final conclusion

This implementation provides a complete RL solution pipeline for the assignment: a stochastic Super Tic-Tac-Toe simulator, validated rules, a trained full-board agent, convergence evidence, opponent-based evaluation and interpretable diagnostics. It demonstrates a meaningful RL agent without claiming theoretical optimality.

The same notebook can regenerate the training curve, evaluation table and diagnostic plots from the saved random seed and experiment settings.